

GAE-Advantage와 Value Model을 활용한 Token-Weighted MTP (WMTP)

DeepSix - 김상호, 김은경, 원우식, 정이우, 한영재, 홍혜진

2025년 12월 17일

Abstract

최근 대규모 언어 모델(LLM)의 추론 효율성을 높이기 위해 Multi-Token Prediction(MTP)이 제안되었으나, 기존 방식은 모든 미래 토큰에 동일한 학습 가중치를 부여한다는 한계가 있다. 코드 생성과 같이 정답과 오답의 분기가 명시적으로 갈리는 도메인에서는 이러한 균등 학습이 비효율적일 수 있다. 본 연구는 이러한 문제를 해결하기 위해 **Weighted MTP (WMTP)**를 제안한다. WMTP는 독립적인 Critic 모델을 통해 토큰별 가치를 추정하고, GAE(Generalized Advantage Estimation)를 사용하여 토큰의 상대적 중요도(Advantage)를 계산한다. 이를 바탕으로 AWR(Advantage-Weighted Regression) 스타일의 가중치를 MTP 손실 함수에 적용함으로써, 모델이 결정적인 토큰에 더 집중하여 학습하도록 유도한다. 실험 결과, 전체 데이터셋의 5.4%(200K 샘플)만을 사용하여 학습했음에도 불구하고, Out-of-Domain 벤치마크에서 HumanEval Pass@20(+5.00%p), MBPP Pass@20(+5.00%p)의 일관된 향상을 보였다. 특히 GSM8K에서는 Pass@20 기준 +4.00%p(30.77% 상대 향상)를 기록하여, 다양성 기반 추론 시나리오(Best-of-N)에서 강점을 보였다. 본 보고서는 WMTP의 이론적 배경, 구현 방법, 그리고 실험 결과를 상세히 기술하고 현재 접근법의 한계와 향후 연구 방향을 논의한다.

1 서론

Meta가 제안한 Multi-Token Prediction(MTP) 아키텍처는 한 번에 하나의 토큰만을 예측하는 기존 방식(Next Token Prediction, NTP)과 달리, 여러 개의 미래 토큰을 동시에 예측함으로써 추론 속도와 학습 효율성을 동시에 개선하고자 했다. MTP는 모델이 더 넓은 문맥을 고려하도록 유도하여 전반적인 코드 품질을 향상시키는 데 기여한다. Qwen3-Next, DeepseekR1 등 프론티어 모델의 아키텍처로 활용된다.

그러나 기존 MTP 방식은 모든 미래 토큰의 예측 오차를 단순히 평균 내어 학습한다는 구조적 한계를 가진다. 코드 생성 과제에서 모든 토큰은 동등한 중요도를 갖지 않는다. 예를 들어, 반복되는 변수 선언이나 괄호 닫기와 같은 문법적 토큰보다는, 알고리즘의 핵심 로직을 결정하는 함수명, 조건문의 분기 조건, 또는 반환값과 같은 토큰이 프로그램의 정확성에 훨씬 결정적인 영향을 미친다. 표준 MTP의 균등한 손실 함수(Uniform Cross-Entropy Loss)는 이러한 토큰 간의 중요도

차이를 반영하지 못하며, 결과적으로 모델이 정말 중요한 결정 지점을 학습하는 데 충분한 자원을 집중하지 못하게 만든다.

본 연구는 "모든 토큰이 동등하게 중요하지 않다"는 가설에서 출발한다. 우리는 정답 코드 생성에 기여도가 높은 토큰에 더 높은 가중치를 부여하여 학습한다면, 동일한 데이터와 연산 자원으로도 더 높은 성능을 달성할 수 있을 것이라 가정했다. 이를 구현하기 위해 강화학습(RL)의 가치 평가 방법론을 지도 학습(Supervised Learning)에 접목한 Weighted MTP(WMTP)를 제안한다.

WMTP는 두 단계로 구성된다. 첫째, 정답 코드와 오답 코드를 구별할 수 있는 독립적인 Value Model(Critic)을 학습한다. 둘째, 학습된 Critic을 사용하여 각 토큰의 가치(Value)를 추정하고, 이를 바탕으로 토큰별 학습 가중치를 동적으로 계산하여 Policy Model을 학습한다. 이 과정에서 로봇 제어 분야에서 활용되던 GAE(Generalized Advantage Estimation)를 도입하여 토큰의 기여도를 안정적으로 추정하고, Bradley-Terry 모델 기반의 Pairwise Ranking Loss를 통해 Critic이 코드의 상대적 우열을 효과적으로 학습하도록 설계했다.

본 보고서에서는 WMTP의 상세한 아키텍처와 학습 방법론을 설명하고, CodeContests 데이터셋을 활용한 실험 결과를 분석한다. 특히 적은 양의 데이터로 학습했음에도 불구하고 다양한 외부 벤치마크(Out-of-Domain)에서 관찰된 성능 향상의 의미를 고찰하고, 동시에 내부 벤치마크(In-Domain)에서의 성능 하락 원인을 분석하여 향후 연구 방향을 제시한다.

2 관련 연구

본 연구는 오프라인 강화학습(Offline RL)과 LLM의 미세 조정(Fine-tuning) 기술을 융합한 시도이다. WMTP의 핵심 아이디어에 영향을 준 주요 연구들은 다음과 같다.

가장 직접적인 이론적 토대는 **AWR(Advantage-Weighted Regression)**이다. Peng et al.(2019)이 제안한 AWR은 오프라인 강화학습에서 Policy Gradient의 높은 분산 문제를 해결하기 위해 제안되었다. AWR은 행동의 가치(Advantage)가 높을수록 해당 행동을 할 확률을 지수적(Exponential)으로 높이는 방식으로 지도 학습을 수행한다. 즉, $\exp(A/\beta)$ 형태의 가중치를 Cross-Entropy Loss에 적용한다. WMTP는 이 가중치 적용 방식을 차용하되, 로봇 제어 도메인에 맞춰진 AWR의 Advantage 계산 방식을 코드 생성 도메인에 맞게 GAE로 대체하고, Value 학습 방식을 Pairwise Ranking으로 변경하여 적용했다.

LLM에 오프라인 강화학습을 적용하려는 시도로는 **ILQL(Implicit Language Q-Learning)**과 **DPO(Direct Preference Optimization)** 등이 있다. ILQL은 Value Function을 학습하여 추론 시점에 Logit을 조정하는 방식을 사용하지만, 이는 추론 비용을 증가시키는 단점이 있다. DPO는 Reward Model 없이 선호도 데이터를 직접 학습에 활용하여 RLHF(Reinforcement Learning from Human Feedback)를 단순화했으나, 토큰 단위의 세밀한 가중치 조절보다는 시퀀스 전체의 확률을 조정하는 데 초점을 맞춘다. 반면, WMTP는 학습 시점에만 Value Model을 사용하여 토큰별 가중치를 계산하고, 추론 시에는 원본 모델 구조를 그대로 유지하므로 추가적인 연산 비용이 발생하지 않는다.

Multi-Token Prediction(MTP) 연구인 Gloeckle et al.(2024)은 여러 토큰을 동시에 예측하는 아키텍처를 통해 LLM의 성능을 개선했다. 하지만 앞서 언급했듯 토큰별 중요도를 고려하지 않는다는 한계가 있었다. 최근 **Rho-1**과 같은 연구들이 토큰 선별(Token Selection)의 중요성을 강조하고 있으나, WMTP는 단순히 토큰을 선택/배제하는 것을 넘어, 연속적인 가중치를 부여함으로써 학습 신호의 강도를 세밀하게 조절한다는 점에서 차별화된다.

3 제안 방법: Weighted MTP

WMTP는 크게 두 단계의 파이프라인으로 구성된다. 첫 번째 단계(Phase 1)에서는 코드의 정/오답 라벨을 보상 신호(Reward Signal)로 활용해, 토큰 단위로 정답과 오답을 예측하는 Critic(Value Model)을 학습하고, 두 번째 단계(Phase 2)에서는 Critic이 제공하는 신호를 바탕으로 Policy Model(MTP)을 가중치 기반으로 학습한다.

3.1 Phase 1: Pairwise Ranking을 통한 Critic 학습

코드 생성 모델의 출력을 평가하기 위해서는 각 토큰이 최종 정답 생성에 얼마나 기여하는지를 판단할 수 있어야 한다. 이를 위해 우리는 독립적인 Value Model을 학습시킨다. Value Model에게 정확한 반사실(Counter Factual) 정보를 제공하기 위해 동일한 Problem Id에 대한 정/오답 샘플 Pair를 제공하여 학습한다.

3.1.1 Value Model 구조

Value Model은 2.7B 크기의 Sheared-LLaMA를 기반으로 한다. 이 모델은 코드 데이터에 대한 사전 학습이 되어 있어 코드의 문맥을 잘 이해한다. 또한, MTP 모델과 동일한 Tokenizer를 공유하므로, 토큰 단위 가중치를 MTP 모델이 이어받아 학습할 수 있다.

모델의 마지막 층에는 MLP(Multi-Layer Perceptron) Head를 부착하여, 각 토큰 위치에서 스칼라 값인 Value V_t 를 출력하도록 설계했다. 이는 해당 시점까지의 코드가 주어졌을 때, 최종적으로 정답 코드가 완성될 기대치를 의미한다.

3.1.2 Pairwise Ranking Loss

초기 연구 단계에서는 일반적인 가치 학습과 같이 MSE(Mean Squared Error)를 사용하여 특정 점수를 예측하도록 시도했으나, Validation Loss가 감소하지 않고 오히려 상승하는 등 학습이 제대로 이루어지지 않았다. 반면, Pairwise Ranking Loss를 도입하자 Train 및 Validation Loss가 안정적으로 감소하며 최종적으로 Validation Accuracy 0.63을 달성했다.

InstructGPT의 Reward Model은 인간 피드백을 통해 실수 범위의 풍부한 점수를 제공하는 반면, CodeContests 데이터셋은 1(정답) 또는 0(오답)이라는 단순한 이진 신호만을 제공한다. 이러한

데이터 특성상 절대적인 점수를 추정하는 MSE보다는, 정답과 오답 간의 상대적 우열을 학습하는 Pairwise 방식이 훨씬 효과적이었던 것으로 분석된다.

따라서 우리는 Bradley-Terry 모델에 기반한 Pairwise Ranking Loss를 채택했다. 동일한 문제에 대해 정답 코드(y^+)와 오답 코드(y^-) 쌍을 구성하고, 정답 코드의 평균 Value가 오답 코드보다 높게 나오도록 학습한다.

$$\mathcal{L}_{\text{rank}} = -\log \sigma(\bar{V}(y^+) - \bar{V}(y^-)) \quad (1)$$

여기서 $\bar{V}(y)$ 는 시퀀스 내 토큰들의 평균 Value이다. 이 방식은 Value의 절대적인 스케일에 의존하지 않고 두 코드 간의 우열 관계를 학습하므로, 학습 과정이 훨씬 안정적이며 오답 데이터(Negative Sample)를 학습에 명시적으로 활용할 수 있다는 장점이 있다.

3.1.3 데이터 샘플링 전략

효과적인 Critic 학습을 위해 데이터 샘플링에도 주의를 기울였다. 단순히 무작위로 쌍을 추출할 경우, 특정 문제에 데이터가 편중되거나 코드 길이에 따른 편향이 발생할 수 있다. 이를 방지하기 위해 **Length-Balanced Stratified Sampling**을 적용했다. 정답과 오답 코드의 길이가 유사한 쌍을 우선적으로 추출하여, Critic이 단순히 "길이가 짧은 코드가 정답"이라는 식의 잘못된 편향(Spurious Correlation)을 학습하지 않고 코드의 내용에 집중하도록 유도했다.

3.2 Phase 2: GAE 기반 가중치 계산 및 Policy 학습

Critic 학습이 완료되면, 이를 사용하여 Policy Model(MTP)을 학습한다. 이 단계에서 Critic은 가중치(Freeze)가 고정된 상태로 사용된다. 지도학습을 진행하므로 오답 샘플은 사용하지 않고 정답 샘플만 사용해서 학습한다.

3.2.1 GAE를 이용한 Advantage 추정

Policy Model이 생성하는 각 토큰이 얼마나 좋은지 판단하기 위해 Advantage를 계산한다. 우리는 강화학습에서 널리 쓰이는 **GAE(Generalized Advantage Estimation)**를 도입했다.

먼저 각 시점 t 에서의 TD-Error(Temporal Difference Error) δ_t 를 계산한다. RLHF의 표준을 따라 할인율(Discount Factor) γ 은 1.0으로 설정한다.

$$\delta_t = V(s_{t+1}) - V(s_t) \quad (2)$$

δ_t 는 토큰 t 가 생성됨으로써 변한 가치의 양, 즉 해당 토큰의 한계 기여분을 의미한다. GAE는 이 TD-Error를 미래 시점까지 누적하여 Advantage A_t 를 계산한다.

$$A_t = \sum_{l=0}^{\infty} (\lambda)^l \delta_{t+l} \quad (3)$$

여기서 λ 는 InstructGPT를 포함한 주요 RLHF PPO 연구에서 채택한 표준 값인 0.95를 차용했다. λ 가 0이면 현재 토큰의 즉각적인 가치 변화만 고려하고, 1이면 에피소드 끝까지의 모든 변화를 합산(Monte Carlo)한다. 0.95는 이 둘 사이의 균형을 잡아, 편향(Bias)과 분산(Variance)을 적절히 조절하여 안정적인 Advantage 추정을 가능하게 한다. 또한 Policy와 파라미터를 공유하지 않는 독립적인 Value Model(Critic)을 별도로 운용하는 구조 역시 InstructGPT의 방식을 따르는 것이다.

3.2.2 AWR 스타일의 가중치 변환

계산된 Advantage A_t 를 학습 가중치 w_t 로 변환한다. 이 과정에서 AWR의 핵심 아이디어인 지수 변환을 사용한다.

$$w_t = \exp\left(\frac{A_t - \mu}{\sigma + \epsilon} \cdot \frac{1}{\beta}\right) \quad (4)$$

Advantage 값의 분포가 배치마다 다를 수 있으므로, 배치 내 통계량(μ, σ)을 이용해 정규화(Whitening)를 수행한다. 개별 토큰의 가치가 단순히 하나의 문장, 혹은 하나의 배치 내에서 평가되지 않고, 전체 샘플에 대해 정규화될 수 있도록 이동 평균(EMA)으로 통계량을 관리하며, 계산된 가중치는 $[0.1, 3.0]$ 범위로 클리핑(Clipping)하여 극단적인 가중치로 인한 학습 불안정을 방지했다. β 는 온도 상수로, 값이 작을수록 좋은 토큰과 나쁜 토큰의 가중치 차이를 크게 벌린다.

IQL과 같은 Offline RL 연구에서는 보통 β 를 0.3 내외로 공격적으로 설정하여 Advantage가 높은 데이터에 집중한다. 그러나 본 연구의 실험 분석 결과, 학습된 Value Model의 Value값 표준편차(std)가 약 1.9로 매우 크게 형성됨을 확인했다. 이 상황에서 낮은 β 를 사용하면 가중치가 폭발적으로 증가하여 학습이 불안정해질 위험이 있다. 따라서 우리는 $\beta = 1.0$ 으로 보수적으로 설정하여 가중치 분산을 제어하고 학습의 안정성을 확보했다.

3.2.3 Weighted MTP Loss

최종적으로 계산된 가중치 w_t 를 MTP의 손실 함수에 적용한다. MTP는 K 개의 미래 토큰을 예측하므로, 각 Head의 손실에 해당 시점의 가중치를 곱하여 합산한다.

$$\mathcal{L}_{\text{WMTP}} = \frac{1}{K} \sum_{k=1}^K \sum_t w_t \cdot \text{CE}(z_{t,k}, y_{t+k}) \quad (5)$$

이 손실 함수를 통해 Policy Model은 가치가 높은(Advantage가 큰) 토큰, 즉 정답 생성에 결정적인 기여를 하는 토큰을 더 정확하게 예측하도록 집중적으로 학습된다. 반면, 가치가 낮은 토큰에 대해서는 상대적으로 작은 페널티를 받게 된다.

4 실험 설정 및 결과

제안한 WMTP의 효용성을 검증하기 위해 CodeContests 데이터셋을 사용하여 모델을 학습시키고, 다양한 코드 생성 벤치마크에서 성능을 평가했다.

4.1 실험 환경

데이터셋: CodeContests 데이터셋의 학습 셋 중 약 5.4%에 해당하는 200,000개의 샘플만을 사용하여 학습 효율성을 검증했다. Critic 학습에는 200,000개의 정답/오답 쌍을 사용했다.

모델: Policy Model로는 Meta-LLaMA2 7B 모델에 MTP Head($K=4$)를 부착하여 사용했으며, Critic Model로는 Sheared-LLaMA 2.7B를 사용했다. 연산 자원의 효율성을 위해 두 모델 모두 LoRA(Low-Rank Adaptation)를 적용하여 미세 조정했다. Policy Model은 $\text{rank}=64$, $\alpha=128$ 로 설정하여 trunk 32개 레이어와 MTP extra head 3개 레이어를 포함한 총 35개 레이어에 LoRA를 적용, 약 175M개(전체 7B의 2.5%)의 학습 파라미터를 사용했다. Critic Model은 과적합 방지를 위해 $\text{rank}=32$, $\alpha=64$ 로 축소하여 약 52M개(전체 2.7B의 1.9%)로 구성했다.

인프라: NVIDIA H100 80GB GPU 4대를 사용하여 분산 학습(FSDP, ZeRO-3)을 수행했다.

학습 설정: Baseline MTP와 WMTP의 주요 하이퍼파라미터는 다음과 같다: 학습률 $1e-4$, 배치 크기 96(effective), 최대 시퀀스 길이 2048, cosine 스케줄러(warmup 3%), 1 epoch 학습. WMTP 고유의 강화학습 하이퍼파라미터로 GAE $\lambda=0.95$, $\beta=1.0$, weight clipping $[0.1, 3.0]$ 을 사용했다. 두 모델은 동일한 기본 설정 하에서 학습하여 가중치 학습 기법의 효과를 공정하게 비교할 수 있도록 했다.

평가 방법: 각 벤치마크에 대해 다음과 같은 평가 방식을 적용했다:

- **HumanEval** ($n=80$): 함수 실행 테스트
- **MBPP** ($n=100$): Assert 문 실행 테스트
- **GSM8K** ($n=100$): 최종 답안 exact match
- **CodeContests** ($n=20$): stdin/stdout 매칭 테스트

Temperature 선택: 모든 모델(Pure, Baseline, WMTP) × 모든 벤치마크 조합에 대해 Temperature 0.2와 0.8을 모두 평가하여, 각 모델별로 Pass@1 기준 최고 점수를 기록한 Temperature를 선택했다. 결과적으로 HumanEval에서는 $T=0.8$ 이, 나머지 벤치마크(MBPP, GSM8K, CodeContests)에서는 $T=0.2$ 가 최적으로 나타났다.

CodeContests 평가 기준: CodeContests의 원본 벤치마크는 문제당 평균 30개 이상의 테스트 케이스를 포함하며, 모든 테스트를 통과해야 정답으로 인정한다. 그러나 본 연구에서는 연산 효율성을 위해 무작위로 샘플링된 10개의 테스트 케이스만을 사용하여 평가를 수행했다. 이는 평가 시간을 단축하면서도 통계적으로 유의미한 비교가 가능하도록 설계된 절충안이다. 공정한 비교를

위해 모든 모델(Pure, Baseline, WMTP)에 동일한 seed(42)로 샘플링된 동일한 테스트 케이스를 사용했다.

4.2 평가 결과

학습된 모델을 HumanEval, MBPP, GSM8K(수학 추론), 그리고 CodeContests 테스트 셋에 대해 평가했다. 비교 대상으로 사전학습된 MTP 모델(Pure), 가중치 없이 일반적인 MTP SFT로 학습한 모델(Baseline), 그리고 제안한 WMTP(Verifiable)를 포함하여 총 세 가지 모델을 평가했다. Pass@K 지표는 Chen et al.(2021)의 unbiased estimator를 사용하여 문제당 20개의 샘플에서 계산했다.

Table 1: 코드/수학 생성 벤치마크 Pass@K 정답률 평가

Dataset	Model	Pass@1	Pass@5	Pass@10	Pass@20
HumanEval	Pure	2.94%	13.17%	23.30%	37.50%
	Baseline	2.56%	11.85%	21.45%	35.00%
	WMTP	3.25%	14.27%	24.79%	40.00%
MBPP	Pure	17.00%	24.58%	27.99%	31.00%
	Baseline	17.50%	23.55%	26.28%	30.00%
	WMTP	17.80%	26.26%	30.29%	35.00%
GSM8K	Pure	2.95%	7.34%	10.40%	14.00%
	Baseline	3.20%	8.80%	11.34%	13.00%
	WMTP	3.00%	8.45%	12.16%	17.00%
CodeContests	Pure	0.25%	1.25%	2.50%	5.00%
	Baseline	1.00%	4.25%	6.97%	10.00%
	WMTP	0.50%	2.50%	5.00%	10.00%

Table 2: WMTP vs Baseline 개선율 ($\Delta = \text{WMTP} - \text{Baseline}$)

Dataset	Domain	$\Delta\text{Pass@1}$	$\Delta\text{Pass@5}$	$\Delta\text{Pass@10}$	$\Delta\text{Pass@20}$
HumanEval	Out-of-Domain	+0.69%p	+2.42%p	+3.34%p	+5.00%p
MBPP	Out-of-Domain	+0.30%p	+2.71%p	+4.01%p	+5.00%p
GSM8K	Out-of-Domain	-0.20%p	-0.35%p	+0.82%p	+4.00%p
CodeContests	In-Domain	-0.50%p	-1.75%p	-1.97%p	0.00%p

4.2.1 벤치마크 성능 비교

Table 1에서 볼 수 있듯이, **Out-of-Domain** 벤치마크인 HumanEval과 MBPP에서 WMTP는 모든 Pass@K 지표에서 Baseline과 Pure를 상회하는 성능을 보였다. 특히 Pass@20에서 HumanEval은 5.00%p, MBPP는 5.00%p의 향상을 기록하여, 샘플링 기반 추론(Best-of-N) 시나리오에서 WMTP의 효과가 두드러짐을 확인했다.

GSM8K의 경우, Pass@1에서는 소폭 하락(-0.20%p)했으나 Pass@20에서 4.00%p(상대적으로 30.77%)의 유의미한 향상을 보였다.

학습 데이터와 동일한 도메인인 **CodeContests** 평가에서는 Pass@1 기준 Baseline(1.00%)이 WMTP(0.50%)보다 높은 성능을 보였다. 그러나 Pass@20에서는 둘 다 동일한 10.00%를 기록하여, 출력한 정답 후보가 많은 경우엔 유사한 성능으로 수렴되는 것을 확인했다. WMTP가 In-Domain에서 성능이 상대적으로 낮은 이유에 대한 분석은 다음 장에서 다룬다.

4.2.2 Phase 2 학습 중 Weight 통계 지표

Table 3: WMTP 학습 중 주요 통계 지표

지표	값	의미
weight_mean	1.328	평균적으로 Baseline 대비 약 33% 더 큰 손실 부여
weight_std	0.981	토큰별 가중치의 분산이 큼 (중요도 구분이 뚜렷함)
clipping_ratio	17.9%	전체 토큰의 약 17.9%가 가중치 상/하한에 도달함

Table 3는 Phase 2 학습 과정에서 기록된 가중치(Weight)의 통계적 특성을 보여준다. 평균 가중치(weight_mean)가 1.0을 상회한다는 것은 모델이 전체적으로 Baseline보다 더 강한 학습 신호를 받고 있음을 의미한다. 다만, AdamW와 같은 적응형(Adaptive) Optimizer는 그래디언트의 크기를 정규화하여 업데이트에 반영하므로, 전체적인 Loss 스케일의 상승(33%) 자체가 학습률을 단순히 높이지 않는다. 즉, Optimizer의 정규화 효과로 인해 전체 토큰에 대한 일괄적인 Loss 상승분은 상쇄되고, 결과적으로 토큰 간의 '상대적인 가중치 차이'만이 학습의 방향성을 결정하는 핵심 요인이 된다. 또한, 약 17.9%의 토큰이 클리핑(Clipping)되었다는 점은, 모델이 일부 토큰을 '매우 중요하거나' 혹은 '매우 불필요한' 것으로 확실하게 구분하고 있음을 시사한다. 이러한 가중치의 동적인 할당이 결과적으로 모델의 일반화 성능 향상에 기여했을 것으로 분석된다.

5 분석 및 토의

5.1 Out-of-Domain 전이 성능의 의미

WMTP가 HumanEval과 MBPP에서 보인 성능 향상은 주목할 만하다. CodeContests는 난이도가 매우 높은 경쟁 프로그래밍 문제들로 구성되어 있는 반면, HumanEval과 MBPP는 상대적으로 짧고 독립적인 함수 단위의 문제들이다. WMTP가 학습 과정에서 "중요한 토큰"을 식별하는 능력을 길렀고, 이것이 문맥은 다르지만 코드 구조가 명확한 외부 벤치마크 문제들을 해결하는 데 도움을 주었을 것으로 해석된다. 이는 가중치 학습이 모델이 불필요한 패턴을 암기(Overfitting)하는 것을 방지하고, 코드의 핵심 구조를 학습하도록 유도하는 정규화(Regularization) 효과를 주었음을 시사한다. 또한 200K 샘플(전체의 5.4%) 학습만으로 이러한 성능 향상을 이끌어낸 것은, 가중치 학습이 데이터 효율성(Data Efficiency)을 크게 높여줄 수 있다는 가능성을 보여준다.

5.2 In-Domain 성능 열위의 원인 분석

CodeContests에서의 성능이 Baseline에 비해 낮은 것은 **제한된 학습 데이터와 가중치 학습의 특성**이 복합적으로 작용한 결과로 해석된다.

본 연구는 전체 데이터의 약 5.4%만을 사용하여 학습을 진행했다. WMTP는 가중치를 통해 일부 토큰에 집중하고 나머지 토큰은 사실상 스킵하는 효과를 낸다. 데이터가 충분할 때는 이러한 선택과 집중이 효율적이지만, 데이터가 극히 적은 상황에서는 모델이 In-domain 문제의 복잡한 패턴을 꼼꼼히 학습할 기회를 놓치게 되어, 모든 토큰을 균등하게 학습하는 Baseline(일반 MTP SFT) 대비 정확도가 하락했을 가능성이 있다.

하지만 주목할 점은 다양성을 고려한 지표인 Pass@20에서는 WMTP와 Baseline이 동일한 10.00%를 기록하며 성능이 수렴한다는 것이다(Table 1). 이는 WMTP가 비록 Top-1 정확도에서는 밀리더라도, **모델의 내재적인 일반화 능력** 자체는 훼손되지 않았거나 오히려 잠재력을 가지고 있음을 시사한다. 즉, 적은 데이터로도 핵심적인 토큰을 선별하여 학습함으로써 Out-of-Domain 전이 성능을 높이는 동시에, In-domain에서도 일정 수준 이상의 다양성을 유지하고 있는 것으로 분석된다.

5.3 Ablation Study: Value Loss의 선택

Critic 학습 시 손실 함수 선택이 성능에 결정적인 영향을 미쳤다. 초기 실험(‘critic-pretrain-linear’)에서 MSE(Mean Squared Error)를 사용하여 절대적인 Value 값을 예측하도록 했을 때는 Validation Accuracy가 **55.4%**에 머물렀으며 학습이 매우 불안정했다. 반면, Pairwise Ranking Loss를 도입하여 정답/오답의 상대적 우열을 학습했을 때는 정확도가 **63.1%(+7.6%p)**로 크게 향상되었고 학습 곡선 또한 안정화되었다.

Table 4: Value Loss 함수에 따른 Critic 성능 비교

Loss	Validation Accuracy	특성
MSE	55.5%	불안정, 절대값 예측의 어려움
Pairwise	63.1%	안정적 , 상대 비교가 코드 도메인에 적합

이는 코드 생성과 같이 정답의 기준이 명확하고 상대적 비교가 중요한 도메인에서는 절대적인 점수 예측보다 순위 학습이 훨씬 효과적임을 입증한다.

5.4 Ablation Study: Advantage 추정 방식 선택

Phase 2에서 토큰 가중치를 계산할 때, 단순 TD-Error($\delta_t = V_{t+1} - V_t$)를 직접 사용하는 방식과 GAE($\lambda = 0.95$)를 사용하는 방식을 비교했다.

TD-Error만 사용한 실험에서는 학습 초기부터 Loss가 불안정하게 진동하며 수렴하지 않는 현상이 관찰되어 약 1 epoch 후 학습을 중단했다. 반면, GAE를 적용한 실험에서는 1 epoch 전체에 걸쳐 Loss가 안정적으로 감소했다.

Table 5: Advantage 추정 방식에 따른 학습 안정성 비교

방식	학습 진행	Epoch Loss 변화
TD-Error only	0.5 epoch 후 중단	1.111 (수렴 실패, 불안정한 진동)
GAE ($\lambda=0.95$)	1 epoch 완료	1.085 $\rightarrow$ 1.002 $\rightarrow$ 0.969 $\rightarrow$ 0.953 $\rightarrow$ 0.943

이 결과는 단일 시점의 TD-Error가 분산이 커서 노이즈로 작용함을 시사한다. GAE는 미래 TD-Error들을 λ -가중 합으로 누적하여 스무딩(Smoothing)된 Advantage를 생성하므로, 학습 신호가 안정화되어 수렴이 가능해진 것으로 분석된다.

5.5 연구의 한계 및 향후 과제

본 연구는 유의미한 성과를 거두었으나, 다음과 같은 몇 가지 한계점을 지니며 이는 향후 연구를 통해 보완되어야 한다.

첫째, **MTP 아키텍처의 필수성 미검증**이다. GAE 기반 가중치 학습이 NTP(Next Token Prediction) 모델에서도 동일한 효과를 낼 수 있는지에 대한 대조 실험이 수행되지 않았다. 따라서 현재의 성과가 MTP 구조 자체의 이점인지, 가중치 학습 기법의 이점인지를 명확히 분리하기 위해서는 NTP 환경에서의 추가 검증이 필요하다.

둘째, **데이터 규모에 따른 결과 패턴의 가변성**이다. 본 실험은 전체 데이터의 약 5.4%만을 사용했다. 데이터 양을 늘려 전체 데이터셋으로 학습할 경우, 현재 관찰된 'In-domain 하락, Out-of-domain 상승'이라는 패턴이 유지될지, 아니면 충분한 데이터 학습으로 인해 In-domain 성능까지 개선될지는 미지수다. 따라서 데이터 스케일링(Data Scaling)에 따른 성능 변화 추이를 확인하는 추가 실험이 필요하다.

셋째, **학습 비용의 증가**다. Phase 2 학습 시 Critic 모델의 Forward Pass가 추가되어 학습 연산량이 약 1.3배 증가한다. 비록 추론(Inference) 시에는 오버헤드가 전혀 없다는 장점이 있지만, 학습 효율성을 더욱 높이기 위한 경량화된 Critic 모델 탐색이 필요하다.

6 결론

본 연구에서는 LLM 코드 생성 모델의 효율적인 학습을 위해 GAE 기반의 토큰 가중치 학습 방법인 Weighted MTP(WMTP)를 제안했다. WMTP는 독립적인 Critic 모델과 Pairwise Ranking, 그리고 GAE를 결합하여 코드 생성에 결정적인 역할을 하는 토큰에 집중하여 학습하도록 설계되었다.

실험을 통해 WMTP가 적은 데이터로도 외부 벤치마크(Out-of-Domain) 성능을 효과적으로 개선할 수 있음을 확인했다. 이는 토큰별 가중치 부여가 모델의 일반화 능력을 향상시키는 유효한 전략임을 시사한다. 비록 긴 시퀀스를 갖는 내부 벤치마크(In-Domain)에서는 한계를 보였으나, 이는 Critic 모델의 성능 개선과 GAE 파라미터 튜닝, 그리고 데이터 확장을 통해 극복 가능할 것으로 기대된다.

향후 연구에서는 NTP(Next Token Prediction) 아키텍처에서도 동일한 가중치 방법론이 유효한

지 검증하고, Critic 모델의 규모를 확장하여 가중치의 신뢰도를 높이는 방향으로 연구를 확장할 계획이다. WMTP는 코드 생성을 넘어 논리적 추론이 필요한 다양한 텍스트 생성 분야로 확장될 잠재력을 가지고 있다.